Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No: 9

Team No: 12

Project Title: Command-Line Argument Passing and Program Execution System

Team Members

Roll Number	Student Name	Signature
2520030316	V.GAUTHAM	
2520030220	G.ROHITH	
2520030049	S.SARVESWARA REDDY	

1. Abstract

Command-Line Argument Passing and Program Execution System is a Linux-based Operating Systems project that demonstrates how a command-line application can accept a program name together with user-provided arguments, create a child process using **fork()**, and replace the child process image with the requested program using an appropriate **exec()** family function. The system preserves the command and argument list so that the newly executed program receives them through the standard **argc** and **argv** interface.

The project focuses on core Operating Systems and system programming concepts such as **process creation, process execution, command-line argument transfer, system calls, error handling, and process synchronization**. It also demonstrates how a shell-like parent process prepares an argument vector, how **execvp()** or a related exec function searches for and launches the requested executable, and how the parent uses **wait()** or **waitpid()** to synchronize with the child.

The proposed solution is a Linux C application that accepts different commands and different argument lists, reports invalid commands or execution failures, and displays the arguments received by the new program. The project is directly aligned with the syllabus topics in **CO-1** (system calls, shell role, command execution journey) and **CO-2** (process creation, execution, synchronization, termination, and process-control pitfalls).

2. Problem Statement

A command-line application may need to launch many different programs while passing each program a user-specified set of arguments. The parent application must correctly convert the user input into the argument-vector format expected by a newly executed process. If the command name, argument order, or termination of the argument list is incorrect, the new program may receive wrong data or fail to execute.

This project addresses the problem of **transferring a command and multiple arguments from a parent process to a newly executed program**. It demonstrates process creation using **fork()**, program replacement using an **exec()** family function, and reception of command-line arguments through **argc/argv**. The system must support different commands with different argument lists and handle invalid commands, missing arguments, and exec failures gracefully.

The parent process waits for the child using **wait()** or **waitpid()**, allowing the execution sequence and exit status to be observed. The project provides a practical view of the Linux process execution path from user command input to the new program receiving its arguments.

3. Objectives

1. To understand how command-line arguments are represented using **argc** and **argv**.
2. To accept a command and multiple arguments from the user in a Linux terminal application.
3. To create a child process using the **fork()** system call.
4. To execute the requested program using a suitable **exec()** family function such as **execvp()**.
5. To verify that the new program receives and displays the command-line arguments correctly.
6. To handle invalid commands, missing arguments, and execution errors using appropriate checks and messages.
7. To synchronize parent and child processes using **wait()** or **waitpid()**.
8. To allow repeated execution of different commands with different argument lists.

4. Proposed Methodology

1. User Input: The program prompts the user to enter a command followed by zero or more arguments. The input is tokenized into an argument array suitable for an exec call.

2. Argument Vector Preparation: The first array element stores the command name, following elements store user arguments, and the final element is set to **NULL** as required by the exec interface.

3. Process Creation: The parent process invokes **fork()** to create a child process. The return value of fork() is checked to distinguish parent, child, and failure cases.

4. Program Execution: In the child process, **execvp()** (or another appropriate exec function) is used to execute the requested command with the prepared argument vector. On success, the child process image is replaced by the new program.

5. Argument Reception: The executed program reads **argc** and **argv** and displays the arguments it received, demonstrating that the command and parameters were transferred correctly.

6. Error Handling: If fork() fails, the parent reports the error. If exec() fails because the command is invalid or inaccessible, the child prints an error message using **perror()** and exits with a non-zero status.

7. Synchronization: The parent process uses **wait()** or **waitpid()** to wait for the child and may display whether the child exited normally or with an error.

8. Repeated Testing: The application is tested with different commands and argument lists, including valid commands, commands with no extra arguments, multiple arguments, and invalid commands.

The methodology demonstrates the complete execution flow: **Read Command → Build argv → fork() → Child exec() → New Program Receives argc/argv → Parent wait()/waitpid() → Report Status**.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	**Purpose in the Project**
Process Creation - fork()	Creates a child process in which the requested command will be executed.
Process Execution - execvp() / exec*()	Replaces the child process image with the requested program and passes the argument vector.
Command-Line Interface - argc, argv	Represents and receives the command-line arguments in the newly executed program.
Error Reporting - perror(), exit()	Reports fork/exec failures and terminates the child with an appropriate status.
Process Synchronization - wait() / waitpid()	Allows the parent to wait for the child and observe completion or failure.
Exit Status - WIFEXITED(), WEXITSTATUS()	Lets the parent interpret how the child process terminated.
Shell / PATH-based Command Execution	Demonstrates the syllabus concept of how user-space commands are resolved and executed.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030316	V.GAUTHAM	Command input handling, tokenization, and creation of the argument vector (argv).
2520030220	G.ROHITH	Process creation using fork(), program execution using exec(), and process synchronization.
2520030049	S.SARVESWARA REDDY	Error handling, testing, debugging, integration, and project documentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the environment for process creation, command execution, testing, and process-status observation.
C Programming Language	Used to implement input parsing, argv construction, fork(), exec(), and synchronization logic.
GCC Compiler	Compiles the C source programs into executable binaries.
POSIX Process APIs	Provides fork(), exec*(), wait()/waitpid(), exit-status macros, and related process services.
Linux Terminal / Shell	Used to compile, execute, and test different commands and argument lists.

8. Expected Outcome

The project is expected to produce a functional Linux-based **argument-passing and program-execution system** that demonstrates how a parent application launches requested programs with user-provided command-line arguments.

1. The application will accept a command and multiple arguments from the user.
2. The parent will successfully create a child process using **fork()**.
3. The child will execute the selected program using **execvp()** or another appropriate exec function.
4. The newly executed program will display the **argc** value and each received **argv** entry.
5. Different commands will be executable with different argument lists without changing the parent program.
6. Invalid commands and malformed or missing input will produce clear error messages instead of silent failure.
7. The parent will synchronize with the child using **wait()** or **waitpid()** and can report the child exit status.
8. Overall, the project will provide practical understanding of **process creation, process execution, system calls, shell-style command execution, argument passing, synchronization, and termination** as covered in CO-1 and CO-2.

9. GitHub Repository

GitHub Repository Link: https://github.com/SarveshReddy-07/KLH-CSE-2026-27-12-Command-Argument-Passing-System

Repository Created: Yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: DR. Y.HARIKA DEVI

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: